Slab generation: why FCC metals hit walltime / OOM, and the fix

codes/files/slab_generate.py (SurfaceBuilderWorkChain)

2026-06-15

Contents

1 Symptom **1**

2 Diagnosis **1**
2.1 Benchmark (Ag, `max_index = 2`) 2

3 The fix **2**
3.1 Legitimate oxides are not affected 2
3.2 Verification 2

4 Knobs (in `slab_generate.py`) **2**

5 Not done here (deliberately) **3**

6 Global relax batching (2026-07-21) **3**

1 Symptom

The `SlabGen` CalcJob of `SurfaceBuilderWorkChain` (one job covers all ~10 bulks of a formula) had an **undetermined walltime** and, for plain FCC metals such as **Ag** and **Cu**, was killed either at the 6 h wall or by an **OOM** event — even though those elements have a 1-atom primitive cell and should slab in milliseconds.

2 Diagnosis

It is not the element, it is the *symmetry of the structure that reaches the runner*. The bulks fed to slab generation come out of the generation / CSP / MinimaHopping stage already **MLIP-relaxed**, so they carry ~0.001–0.01 Å of numerical noise (and the generator deliberately explores distorted polymorphs).

`generate_for_structure` standardized with the tight default `symprec=0.01` (`SpacegroupAnalyzer(structure, symprec=0.01, ...)`). At that tolerance a *perfect* FCC metal with a hair of MLIP noise is mis-read as a large **P1** cell. The consequences inside `generate_all_slabs`:

- in P1 **every atomic layer is a distinct termination**, so it enumerates dozens of Miller planes × many spurious shifts;
- with `max_normal_search = max_miller_idx` it builds a **large oriented supercell** for each plane to orthogonalise the c-axis;
- all those slabs, for **all bulks**, are held in RAM in a **single** CalcJob that shares its memory budget with the adsorbate generator/relaxer.

That is the walltime blow-up *and* the OOM.

2.1 Benchmark (Ag, `max_index = 2`)

Input to the runner	spglib sees	`generate_all_slabs`
Clean FCC Ag (Fm-3m, 4-atom conv.)	Fm-3m, 4 at	6 slabs, **3.2 s, 103 MB**
FCC Ag as 2×2×2 + 0.01 Å noise, at `symprec=0.01`	**P1, 32 at**	(pathological)
Same cell, standardized at `symprec=0.05`	Fm-3m, 4 at	6 slabs, **3.2 s**
Genuinely distorted P1, 32 at	P1, 32 at	**TIMEOUT > 180 s**

A clean supercell is *not* the problem — spglib reduces it back. Noise is.

3 The fix

All in `codes/files/slab_generate.py`; the slab metadata contract is unchanged (still a pymatgen `Slab` → `pmg_to_ase`, carrying miller_index / shift / scale_factor / oriented_unit_cell to `slab_relax.py`).

1. **Symprec-ladder standardization** (`_standardize_bulk`). Walk `SYMPREC_LADDER = (0.1, 0.05, 0.01)` (loose → tight) and keep the **smallest** conventional cell **whose reduced composition is preserved**, so a noisy metal collapses back to its 4-atom Fm-3m cell while an alloy is never over-merged into a different stoichiometry. Falls back to the original tight `symprec=0.01` cell if no rung preserves the composition.
2. **`max_normal_search = 1`** (was `= max_miller_idx`). Orthogonality is already enforced afterwards by `process_slab` (it drops any slab with α or β more than 1° off 90°), so a costly normal search is wasted work.
3. **Guards** so one bad cell cannot take down the shared multi-bulk job:
   - **size + symmetry skip** — skip generation only when the standardized cell is **both** large (> `MAX_CONV_ATOMS = 60` atoms) **and** low-symmetry (spacegroup number <= `LOWSYM_SG_MAX = 15`, i.e. triclinic/monoclinic). That is exactly the OOM/hang regime and is not a meaningful slab target.
   - **per-bulk walltime cap** — `generate_all_slabs` runs under a SIGALRM `_time_limit(GEN_TIMEOUT_S = 600 s)`; a slower cell raises `SlabGenTimeout`, which `run_slab_generate` already catches per structure and turns into an empty result (skip), not a job-wide failure.

3.1 Legitimate oxides are not affected

The skip needs *low symmetry* as well as size, so an ordered oxide stays in. A Fd-3m pyrochlore has an 88-atom conventional cell but spacegroup #227 ≫ 15, so it passes; and because it is high-symmetry it has few distinct Miller planes and terminations, so `generate_all_slabs` is well behaved on it anyway.

3.2 Verification

`/tmp/test_slabgen.py` style checks (all pass):

- noisy 32-atom Ag → standardized to 4-atom Fm-3m → 6 slabs in ~2.5 s;
- `_time_limit` raises a catchable `SlabGenTimeout`;
- an AgCu alloy keeps its reduced formula (no over-merge);
- a distorted 108-atom P1 cell (sg #1) is skipped by the guard.

4 Knobs (in `slab_generate.py`)

Constant	Default	Meaning
SYMPREC_LADDER	(0.1, 0.05, 0.01)	loose→tight symprec rungs for standardization
MAX_CONV_ATOMS	60	size half of the skip test
LOWSYM_SG_MAX	15	spacegroup-number half of the skip test
GEN_TIMEOUT_S	600	per-bulk walltime cap on generate_all_slabs

5 Not done here (deliberately)

A lighter, symmetry-free testing backend (ASE ase.build.surface: ~ms per facet, flat memory, immune to input symmetry, but one termination per facet and no oriented-cell metadata) was considered and **not** added — this fix keeps the production pymatgen path and its full termination enumeration.

6 Global relax batching (2026-07-21)

The relax side no longer chunks per bulk (which submitted one under-filled job per bulk: 10 bulks x 12 slabs = 10 jobs of 12). inspect_slabgen now collects ALL generated slabs first and divides them into GLOBAL chunks of <= MAX_SLABS_PER_CHUNK (50): 120 slabs -> 3 jobs of 50/50/20.

Attribution is load-bearing — downstream stages (adsorbates -> photocat gap filter -> manual HSE) identify slabs by the bulk uuid they came from — so mixed-bulk chunks required a payload contract change in slab_relax.py:

- payload items are now {"slab": <encoded>, "uuid", "epa", "index"} — the bulk uuid AND its energy-per-atom ride with each slab (--epa as a per-job cmdline argument is only kept as the legacy bare-list fallback);
- every output record ECHOES uuid + input index; failures are listed per slab in failed_slabs with uuid/index/reason. Attribution is never inferred from output position (non-converged slabs are dropped) or from job topology;
- the workchain regroups chunk outputs by the echoed uuid; the per-bulk top-MAX_NUM_SURF selection is unchanged. A dead chunk reports exactly which bulks lost how many slabs (chunk composition is tracked as counts in the checkpoint — the slabs themselves stay out of it, as before).

Contract-tested in tests/test_slab_relax_contract.py (real runner, ASE EMT: mixed-bulk chunk with a poison entry, per-slab epa actually used, legacy format, missing-epa fails loudly); the chunk-composition bookkeeping is fuzz-verified identical to the actual chunking.